

Relazione Progetto 2 MdCS

Andrea Coldani (894684) Fabio Colonetti (899689)
Francesco Fusi (899741)

Giugno 2026

[Link al codice del progetto](#)

1 Introduzione

L'elaborato descrive l'analisi e lo sviluppo di un sistema software per la compressione di immagini in toni di grigio. Gli obiettivi del progetto si articolano in due fasi principali:

- **Analisi prestazionale:** dedicata all'implementazione in Python dell'algoritmo DCT2 e confronto approfondito con la routine FFT della libreria `scipy`.
- **Compressione JPEG:** sviluppo di un sistema di compressione d'immagine in scala di grigio simile a JPEG.

2 Struttura della libreria

Il software è stato sviluppato con il linguaggio Python e strutturato in modo modulare per garantire separazione tra la logica di calcolo matematico, l'interfaccia utente, i test di validazione. Di seguito viene descritto il ruolo di ciascuna cartella e file presenti nel progetto:

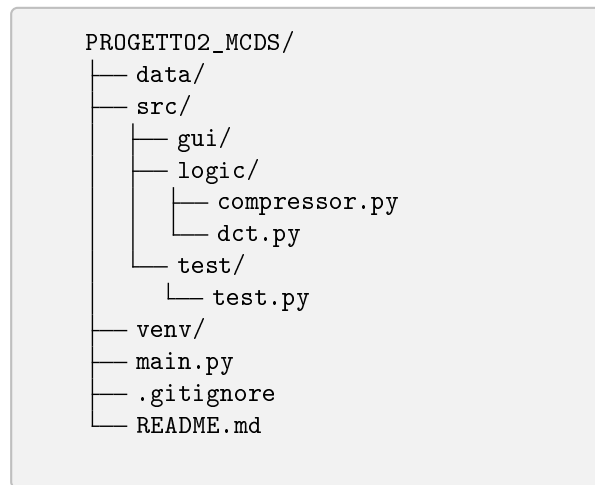


Figura 1: Struttura delle directory del progetto

- **data/**: Cartella contenente le immagini in formato bitmap (**.bmp**) a scala di grigi, utilizzate come input per la compressione.
- **src/** (Source): Contiene il codice sorgente dell'applicazione, suddiviso nei seguenti sotto-moduli specializzati:
 - **gui/**: Contiene il codice che implementa l'interfaccia grafica che consente all'utente di navigare nel filesystem per selezionare l'immagine e inserire i parametri numerici di input (F e d).
 - **logic/**: Contiene il nucleo computazionale dell'applicativo:
 - * **dct.py**: Contiene l'implementazione sequenziale della DCT/-DCT2 definita "fatta in casa" ($O(N^3)$) e le funzioni di interfaccia con la libreria scientifica per la versione "fast" ($O(N^2 \log N)$), necessarie al confronto dei tempi di esecuzione.
 - * **compressor.py**: Contiene la logica dell'algoritmo di compressione JPEG-like, infatti si occupa della segmentazione dell'immagine in macro-blocchi $F \times F$, del filtraggio delle frequenze oltre la diagonale d .
 - **test/test.py**: Contiene lo script di testing utilizzato per verificare la correttezza numerica e lo scaling delle funzioni implementate, confrontando i risultati della DCT monodimensionale e bidimensionale con la matrice di test 8×8 fornita nelle specifiche.
- **main.py**: Il punto di ingresso (*entry-point*) principale dell'applicazione che inizializza l'interfaccia utente e coordina il flusso dei dati tra i vari moduli.

- `venv/`, `.gitignore`, `README.md`: File e directory di configurazione per la gestione dell'ambiente virtuale isolato, l'esclusione di file ridondanti dal controllo di versione e la stesura delle istruzioni rapide per l'installazione e l'esecuzione del programma.

3 Implementazione DCT2

3.1 DCT2

La nostra implementazione si basa sulla definizione classica della DCT2 e sfrutta la proprietà di separabilità della trasformata bidimensionale. Per questo motivo abbiamo prima realizzato una DCT monodimensionale come funzione di base, che per ogni vettore di lunghezza N calcola i coefficienti mediante prodotti scalari con la matrice di trasformazione D .

Nel codice il vettore di ingresso della DCT monodimensionale è chiamato `f_vect`, mentre l'operazione bidimensionale lavora su un blocco quadrato `block` di dimensione $N \times N$.

La matrice di trasformazione D viene costruita in `src/logic/dct.py` nella funzione `compute_D`:

```
D = alpha_vect.reshape(N,1)*np.cos(k*np.pi*(2*i+1)/(2*N))
```

Qui `alpha_vect` contiene i fattori di normalizzazione ortonormali, eseguiti secondo lo scaling ortogonale richiesto dalla specifica:

```
alpha_vect[0] = N**(-0.5)
alpha_vect[1:] = (N**(-0.5)) * np.sqrt(2)
```

mantenendo la trasformata DCT ortogonale. Il termine `np.cos(...)` definisce le basi coseno della DCT.

La DCT2 viene quindi ottenuta applicando due volte la DCT1 in cascata: prima a ciascuna riga del blocco F e poi a ciascuna colonna del risultato intermedio. In altre parole, il calcolo può essere descritto come

$$C = D \cdot F \cdot D^T$$

dove F è la matrice dei valori del blocco e D è la matrice della trasformazione DCT monodimensionale.

Nel codice, la funzione `my_dct1d` esegue il prodotto vettore-matrice

$$\mathbf{c} = D \cdot \mathbf{f}$$

che corrisponde alla riga `c_vect = D @ f_vect`. La DCT2 si ottiene applicando questa funzione prima alle righe e poi alle colonne del blocco.

Poiché ogni prodotto matrice-matrice richiede cicli annidati su strutture dati di dimensione N , l'implementazione diretta segue un costo computazionale teorico dell'ordine di $\mathcal{O}(N^3)$.

3.2 FFT

La libreria che abbiamo utilizzato per calcolare la **DCT** in modo efficiente è **scipy.fftpack**. La **FFT** è una trasformata veloce della **DFT**, quindi ci è sorta spontanea la domanda su cosa centrassero DCT e FFT. Innanzitutto è importante specificare che all'interno di **scipy.fftpack.dct** sono presenti diversi tipi di DCT, noi abbiamo usato la **dctn**, la cui n specifica il fatto che è n -dimensionale. L'API per questa funzione è la seguente:

```
dctn(x, type=2, shape=None, axes=None, norm=None,
      overwrite_x=False)
```

La funzione ha come valore di ritorno y , cioè l'array trasformato. In cui i parametri più importanti sono:

- **x**: vettore di ingresso (nel nostro caso il blocco 2D);
- **type**: tipologia di DCT;
- **norm**: fattore di normalizzazione.

È essenziale scegliere la versione della DCT **type=2**, in quanto è quella che abbiamo visto nel corso, infatti applica la formula:

$$y_k = 2 \sum_{n=0}^{N-1} x_n \cos\left(\frac{\pi k(2n+1)}{2N}\right)$$

Inoltre con il parametro **norm='ortho'**, applica i coefficienti per ortonormalizzare la base:

$$f = \begin{cases} \sqrt{\frac{1}{4N}} & \text{if } k = 0, \\ \sqrt{\frac{1}{2N}} & \text{otherwise} \end{cases}$$

(Nota: in questo paragrafo abbiamo riportato esattamente le formule presenti nella documentazione di SciPy, quest'ultima moltiplica inizialmente y_k per 2, ma poi lo riscalda con i fattori di normalizzazione).

Internamente la libreria per calcolare la DCT (**c_vector**) utilizza la FFT, infatti esiste uno stretto legame tra DFT e DCT, in quanto è possibile rendere il segnale (**f_vector**) simmetrico per far sì che la DFT e la DCT abbiano lo stesso risultato. In particolare la FFT procede in modo ricorsivo con il divide-et-impera, dunque senza calcolare la matrice D dei coefficienti:

$$\begin{aligned} X_k &= \sum_{n=0}^{N-1} x_n \cdot e^{-i2\pi kn/N} \\ &= \sum_{m=0}^{N/2-1} x_{2m} \cdot e^{-i2\pi k(2m)/N} + \sum_{m=0}^{N/2-1} x_{2m+1} \cdot e^{-i2\pi k(2m+1)/N} \\ &= \sum_{m=0}^{N/2-1} x_{2m} \cdot e^{-i2\pi km/(N/2)} + e^{-i2\pi k/N} \sum_{m=0}^{N/2-1} x_{2m+1} \cdot e^{-i2\pi km/(N/2)} \end{aligned}$$

Cioè, si continua a dividere il vettore in ingresso finchè non si raggiunge il caso base che corrisponde a una DFT di lunghezza $N = 2$, in particolare: Per $k = 0$

$$\text{Per } k = 0: \quad X_0 = x_0 \cdot e^0 + e^0 \cdot x_1 \cdot e^0 = x_0 + x_1$$

$$\text{Per } k = 1: \quad X_1 = x_0 \cdot e^0 + e^{-i\pi} \cdot x_1 \cdot e^0 = x_0 - x_1$$

Questa scomposizione riduce il costo computazionale monodimensionale $\mathcal{O}(N \log_2 N)$ (infatti l'algoritmo è molto simile a **merge-sort**). Nel caso 2D il costo totale diventa quindi:

$$\mathcal{O}(N^2 \log_2 N) \quad (1)$$

3.3 Dati Sperimentali Raccolti

La validazione sperimentale è stata condotta su matrici quadrate di dimensione N crescente [128, 256, 512], mediando i tempi su 100 run indipendenti per abbattere la varianza dei tempi di esecuzione.

I risultati, riassunti nelle Tabelle 1 e 2 e il grafico in Figura 2, confermano sperimentalmente i modelli di complessità teorica precedentemente descritti.

N	Tempo Medio Manuale $\mathcal{O}(N^3)$ [s]	Rapporto $T(N)/T(N/2)$
128	1.767253×10^{-1}	—
256	1.771358×10^0	10.02
512	1.404507×10^1	7.93

Tabella 1: Tempi medi di esecuzione nostra implementazione al variare di N .

L'andamento dell'algoritmo implementato da noi mostra una crescita dei tempi $\mathcal{O}(N^3)$. Quando la dimensione della matrice raddoppia, passando da $N = 256$ a $N = 512$, il tempo teorico di esecuzione dovrebbe aumentare di un fattore pari a $2^3 = 8$. Dai dati sperimentali reali si osserva un incremento effettivo estremamente coerente con la teoria, pari a 7.93, confermando quanto detto in precedenza.

Il picco registrato nel passaggio precedente da $N = 128$ a $N = 256$, pari a un rapporto di 10.02 è invece dovuto molto probabilmente all'hardware, in quanto al crescere delle matrici entrano in gioco questioni architetturali relative a cache miss, gestione della memoria e altre ottimizzazioni di basso livello.

N	Tempo Medio Libreria $\mathcal{O}(N^2 \log_2 N)$ [s]	Rapporto $T(N)/T(N/2)$
128	4.061909×10^{-4}	—
256	1.367665×10^{-3}	3.37
512	5.212104×10^{-3}	3.81

Tabella 2: Tempi medi di esecuzione per l'algoritmo di libreria SciPy al variare di N .

Per quanto riguarda l'algoritmo di libreria, la complessità attesa segue $\mathcal{O}(N^2 \log_2 N)$. In questo caso al raddoppiare della dimensione della matrice il

rapporto di crescita teorico non è costante ma decresce al crescere di N . Dunque i rapporti teorici attesi sono pari a:

$$\text{Target Teorico } (128 \rightarrow 256) : \frac{256^2 \cdot \log_2(256)}{128^2 \cdot \log_2(128)} = 4 \cdot \frac{8}{7} \approx 4.57 \quad (2)$$

$$\text{Target Teorico } (256 \rightarrow 512) : \frac{512^2 \cdot \log_2(512)}{256^2 \cdot \log_2(256)} = 4 \cdot \frac{9}{8} = 4.50 \quad (3)$$

Calcolando sperimentalmente, i rapporti sui tempi medi emerge uno scostamento rispetto al modello teorico puro:

$$\text{Rapporto Reale } (128 \rightarrow 256) : \frac{1.367665 \times 10^{-3}}{4.061909 \times 10^{-4}} \approx 3.37 \quad (4)$$

$$\text{Rapporto Reale } (256 \rightarrow 512) : \frac{5.212104 \times 10^{-3}}{1.367665 \times 10^{-3}} \approx 3.81 \quad (5)$$

La differenza tra i valori teorici e quelli sperimentali si spiega analizzando le componenti hardware e software del sistema. In entrambi i casi notiamo che il rapporto reale è inferiore a quello teorico. Questo evidenzia l'efficacia delle ottimizzazioni a basso livello della libreria SciPy (scritta in C/Fortran). A queste dimensioni intermedie, l'algoritmo sfrutta appieno la vettorizzazione dei registri della CPU e il parallelismo hardware, abbattendo i tempi di calcolo oltre la semplice previsione matematica.

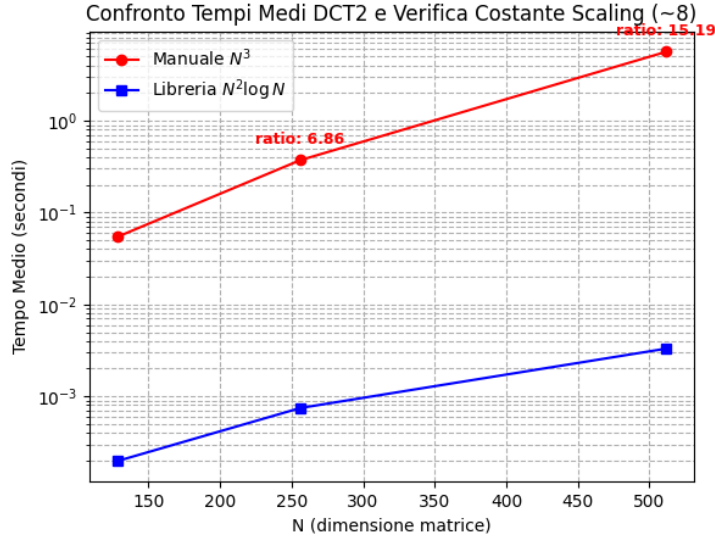


Figura 2: Andamento dei tempi medi di esecuzione al variare della dimensione N .

3.4 Conclusioni

In conclusione possiamo dire che al crescere di N , il divario prestazionale tra i due approcci diventa significativo. Alla dimensione massima testata di $N = 512$, la routine ottimizzata conclude il calcolo in appena 5.21 millisecondi (5.212104×10^{-3} s), risultando circa 2695 volte più rapida dell'algoritmo manuale da noi implementato, il quale richiede invece ben 14.04 secondi (1.404507×10^1 s).

4 Analisi qualità della compressione

Il nostro progetto implementa un algoritmo di compressione ispirato a JPEG. Due parametri principali governano il processo: $\mathbf{F}$, la dimensione dei blocchi, e $\mathbf{d}$, il numero di coefficienti DCT azzerati a partire da quelli ad alta frequenza. Un valore elevato di $\mathbf{d}$ conserva più coefficienti, mentre un valore basso ne elimina di più. Nel JPEG standard si utilizza tipicamente $F = 8$, e il parametro $\mathbf{d}$ si comporta come indice di qualità; nella nostra implementazione entrambi i valori sono invece configurabili liberamente.

Per valutare un'immagine compressa è utile distinguere tra la qualità numerica e quella percettiva. Si possono usare metriche come l'MSE (mean-square-error), il SRN (signal-to-noise ratio) o la differenza pixel per pixel, ma questi indici non descrivono completamente la qualità visiva. Le tecniche di compressione basate sulla DCT eliminano infatti soprattutto le frequenze alte a cui l'occhio umano è meno sensibile, riducendo i dati senza degradare eccessivamente l'impressione visiva dell'immagine.

In questa sezione confrontiamo diverse immagini valutando sia la resa soggettiva sia gli indicatori oggettivi. Abbiamo scelto come misura numerica il RMSE, definito come:

$$RMSE = \sqrt{\frac{1}{MN} \sum_{i=1}^N \sum_{j=1}^M [Y(i, j) - \hat{Y}(i, j)]^2} \quad (6)$$

Il RMSE è intuitivo perché calcola l'errore quadratico medio tra i pixel originali e quelli ricostruiti; a differenza dell'MSE, riporta il risultato sulla stessa scala dei valori di intensità (0-255).

4.1 Variazione del parametro di taglio $\mathbf{d}$

Come prima prova abbiamo usato l'immagine a scacchiera 80x80 e abbiamo fissato la dimensione del blocco a $F = 8$, variando il parametro $\mathbf{d}$, per verificare la variazione di qualità dell'immagine compressa. Abbiamo usato come benchmark per le altre configurazioni un'impostazione standard con $\mathbf{d} = 5$, in cui l'RMSE risulta di 30.55 e la qualità visiva è abbastanza degradata, ma comunque accettabile.

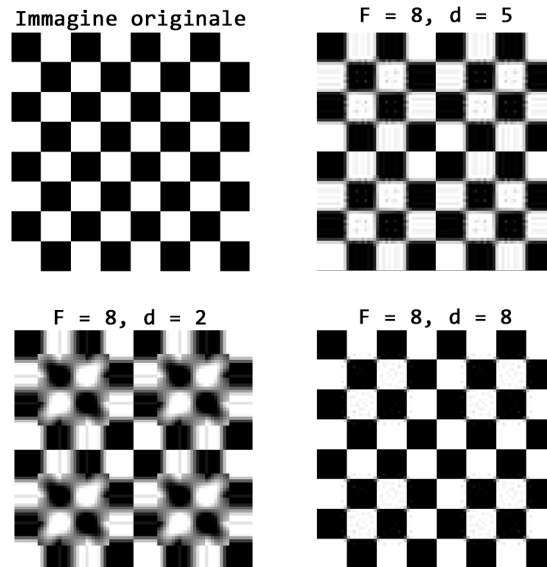


Figura 3: Compressione immagine 80x80.bmp al variare di d

- **Salvando il 50% delle frequenze** ($d \approx 8$) l'RMSE decresce assumendo il valore di 8.89. Dal punto di vista qualitativo, l'immagine risulta praticamente indistinguibile dall'originale. Conservare i coefficienti di ordine superiore introduce nel sistema le alte frequenze spaziali, necessarie per la ricostruzione accurata dei dettagli fini e dei contorni netti.
- **Salvando il 5% delle frequenze** ($d \approx 2$) l'RMSE aumenta a 58.42. Questo porta ai due artefatti tipici della compressione:
 1. *Effetto blocco*: le discontinuità ai confini delle sottomatrici 8×8 diventano marcate.
 2. *Sfocatura*: la rimozione quasi totale delle alte frequenze cancella il gradino bianco nero, rendendo sfocato il passaggio dal bianco al nero.

Un'osservazione interessante è che, impostando $d = 1$, l'immagine risulta suddivisa in blocchi ciascuno dei quali appare come una regione a tonalità di **grigio uniforme**. Questo avviene perché la maggior parte dei coefficienti ad alta frequenza viene annullata dalla quantizzazione, lasciando soltanto la componente DC che corrisponde al valore medio del blocco.

4.2 Analisi Immagini Geometriche

Ora passiamo all'analisi delle immagini "geometriche", dove sono presenti dei blocchi bianchi e neri alternati a scacchiera. In particolare abbiamo valutato

il comportamento della ricostruzione modificando il parametro F a dimensioni superiori ($F = 8, 16, 24, 32$) e mantenendo inalterata la proporzione di taglio al 25%. Proviamo i valori di F e d sull'immagine 160x160bmp:

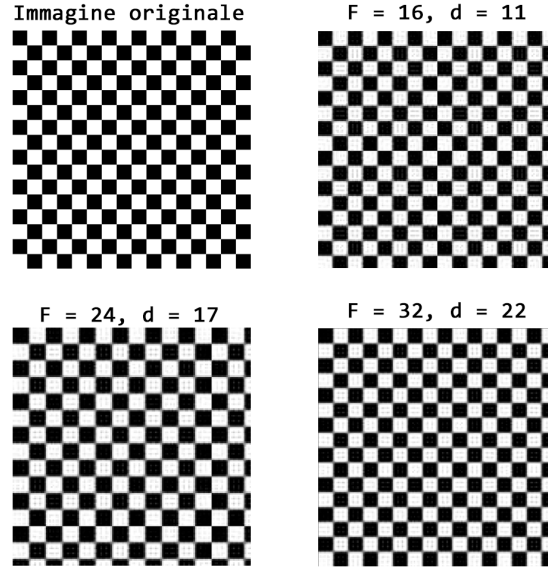


Figura 4: Compressione immagine 160x160.bmp al variare di F

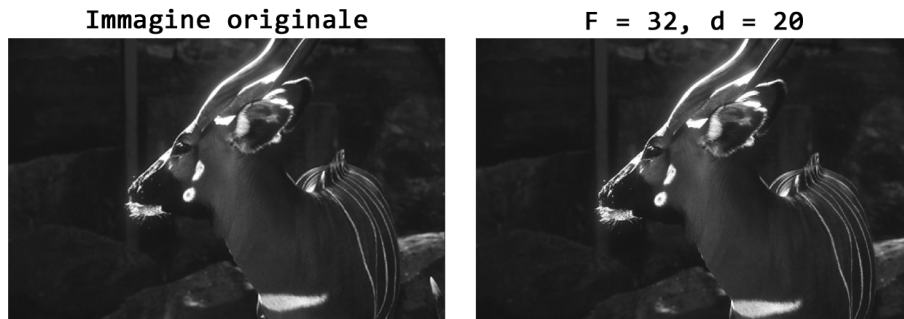
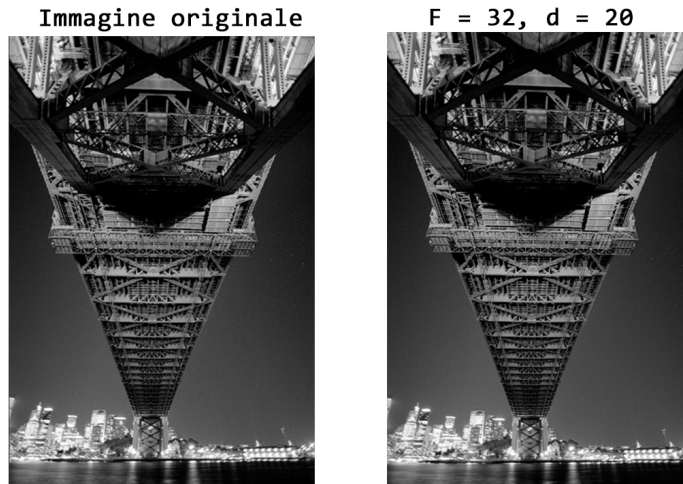
F	d	Percentuale	RMSE
8	5	23.44%	30.55
16	11	25.78%	32.47
24	17	26.56%	164.92
32	22	24.71%	32.42

Tabella 3: RMSE al variare del blocco F , con tasso di taglio a circa 25%.

Questo comportamento evidenzia il legame tra la dimensionalità del blocco F e la periodicità del segnale geometrico quando si applica il taglio diretto della matrice. Infatti nel caso con $F = 24$ (RMSE = 164.92), poiché la dimensione originale dell'immagine (160) non è divisibile per 24, l'operazione di taglio rimuove gli ultimi 16 pixel della matrice ($160 - 24 \times 6 = 16$). Questo troncamento asimmetrico spezza a metà l'ultima fila della scacchiera, aumentando di molto il RMSE. Anche visivamente dalla figura 4, si può notare questo fenomeno: la qualità generale visiva rimane abbastanza invariata per tutte le compressioni, ma nel caso con $F = 24$ il bordo destro risulta troncato in maniera evidente.

4.3 Analisi Immagini Reali

In questa sezione confrontiamo due immagini reali, in particolare quella del cervo (deer.bmp 4.3) e quella del ponte (bridge.bmp 4.3), con parametri $F = 32$ e $d = 20$.



Visivamente è facile notare che l'immagine del ponte è rimasta praticamente invariata, mentre quella del cervo, seppur mantenendo una buona qualità generale, presenta alcuni artefatti nelle zone di alta frequenza (baffi, muso, zone di luce). Sorprendentemente in questo caso il RMSE non riflette questa osservazione, infatti è uguale a 38.53 per il ponte, mentre 33.23 per il cervo. Questo fenomeno è causato dal fatto che il RMSE misura errori pixel-per-pixel e non tiene conto della sensibilità percettiva dell'occhio umano. Piccoli errori concentrati su dettagli strutturalmente importanti (come baffi o contorni) possono risultare molto visibili pur producendo un RMSE contenuto.

4.4 Conclusioni

Provando altre immagini, è facile effettuare alcune osservazioni. L'algoritmo di compressione si comporta in modo "*poco prevedibile*" sulle immagini "matematiche", diversamente da quelle reali, che generalmente mantengono una qualità maggiore. In generale, su tutti i tipi di immagini, è comunque possibile notare che questo algoritmo di compressione è impreciso sulle zone di alta frequenza (cioè dove il colore cambia repentinamente), infatti, ad esempio, nell'immagine del gradiente (gradient.bmp), dato che il cambiamento di colore è graduale e senza variazioni, il RMSE è minimo.

Si può quindi concludere che l'algoritmo a parità di parametri (F e d) non comprime allo stesso modo, ma è molto dipendente dalla struttura dell'immagine.